



RISC-V RV32I Single-Cycle CPU 설계

[HARMAN] 세미콘 아카데미 2기 | 이윤지



목차

01 RV32I



02 TYPE 블록도



03 TYPE 시뮬레이션



04 reference





RISC-V

CPU

프로그램 명령어를 해석하고
실행하는 컴퓨터의 핵심 장치



RISC-V ISA

CPU의 명령어 집합 구조인
ISA중 하나



RV32I

32비트 기반의 정수 명령어
집합으로, 연산·메모리
접근·분기·점프 등 **CPU 동작의**
기본 동작 정의



RV32I

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0		
funct7				rs2		rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type	
imm[11:5]				rs2		rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]		rs2		rs1		funct3		imm[4:1]		imm[11]		opcode		B-type
imm[31:12]										rd			opcode		U-type	
imm[20]		imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type	

표1

RV32I 명령어

- Opcode : type 선택
 - Rd : 결과를 저장할 목적지 레지스터
 - Rs1 : 첫번째 입력 레지스터
 - Rs2 : 두번째 입력 레지스터
- Funct3 : 같은 opcode안에서 세부 동작 구분
 - Funct7 : R-type에서 ADD/SUB, SRL/SRA 연산 구분
 - Imm : 연산 혹은 주소에 직접 사용



CHAPTER 01

RV32I

Register_file

레지스터 쓰기 / 읽기

ROM
Instruction
Memory

inst_rData

inst_Addr

ROM

명령어 저장

Program Counter

다음 명령어 주소 저장

RV32I_Core

Control Unit

Func7 [31:25]

Func3 [14:12]

OPcode [6:0]

Reg_wr_en

JAL

JARL

aluSrcMuxSel

alu_controls

RegWdataSel

d_wr_en

Register File

[19:15]

[24:20]

[11:7]

WA (RD)

WD (RD)

Reg

x0

x1

...

x31

RS1

RS2

Extend

ALU

비교기
(comparator)

b taken

PC

DataPath

Control Unit

명령어 해석, 신호 제어

RAM
Instruction
Memory

wr_en

dAddr

dWdata

dRdata

RAM

데이터 저장 / 로드

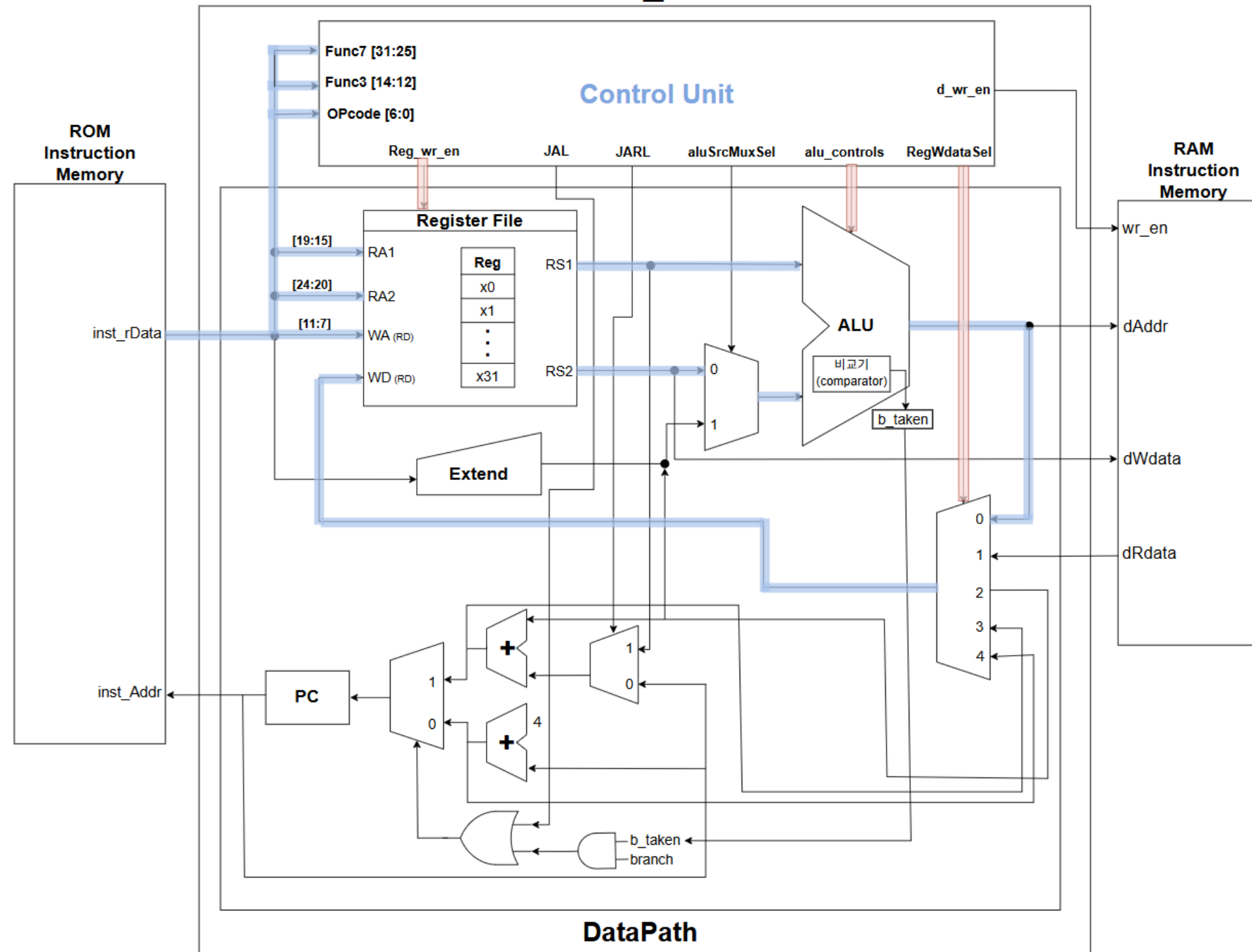


CHAPTER 02



R-TYPE

RV32I_Core

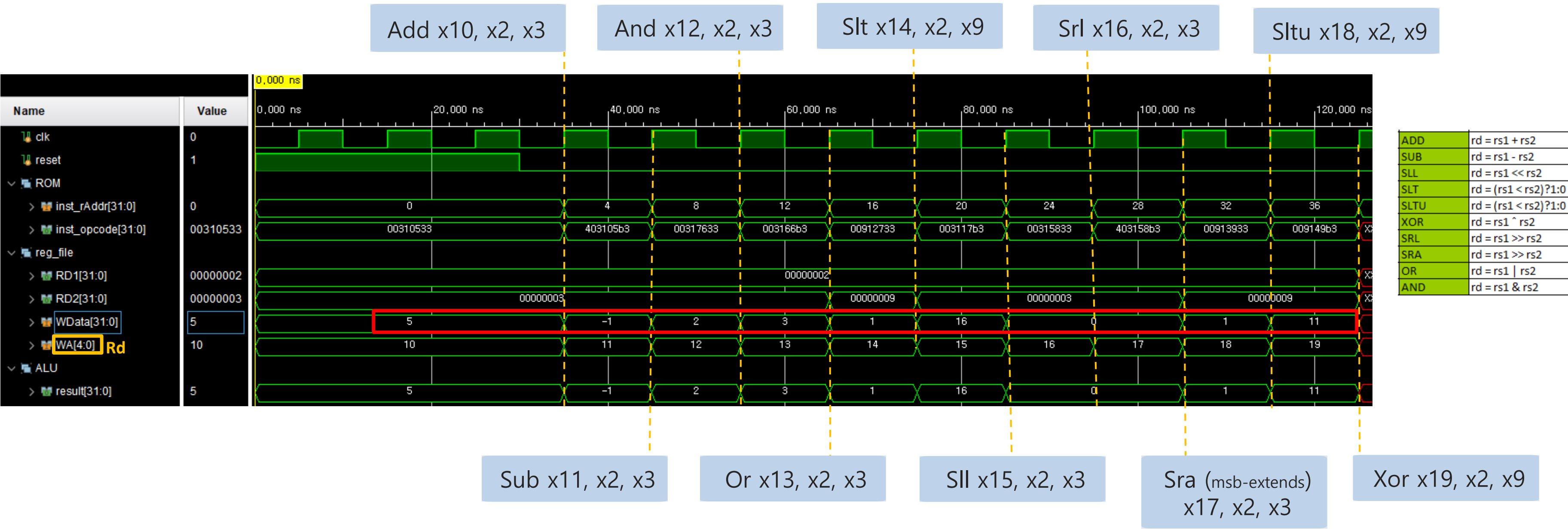


R-TYPE

- 역할 : 레지스터 간 연산을 수행
- 흐름 : Instruction_Memory에서 명령어 받음 >
register_file에서 RD1, RD2 읽음
> ALU 연산
> 결과가 다시 reg_file[rd]에 저장



R-TYPE



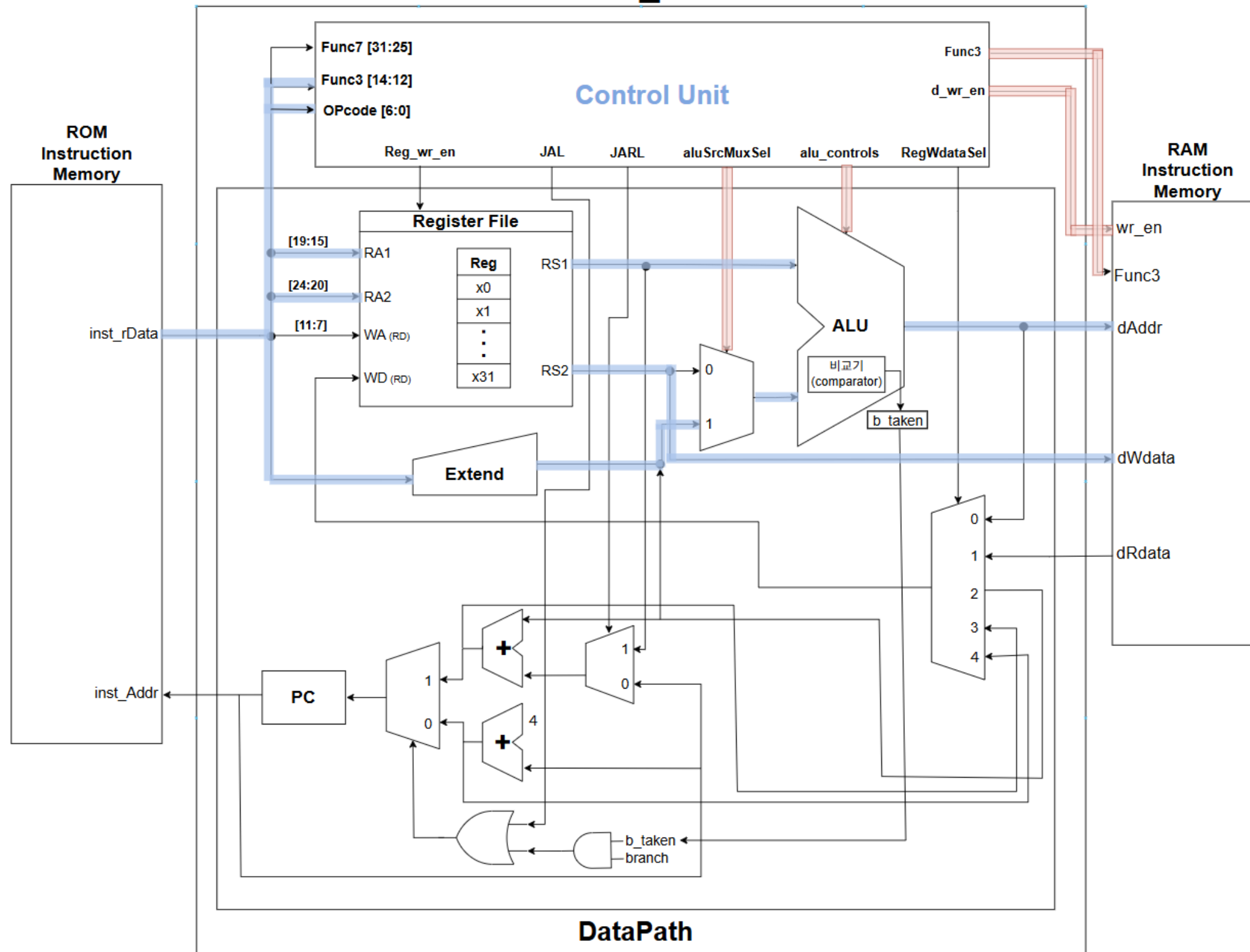


CHAPTER 02



S-TYPE

RV32I_Core



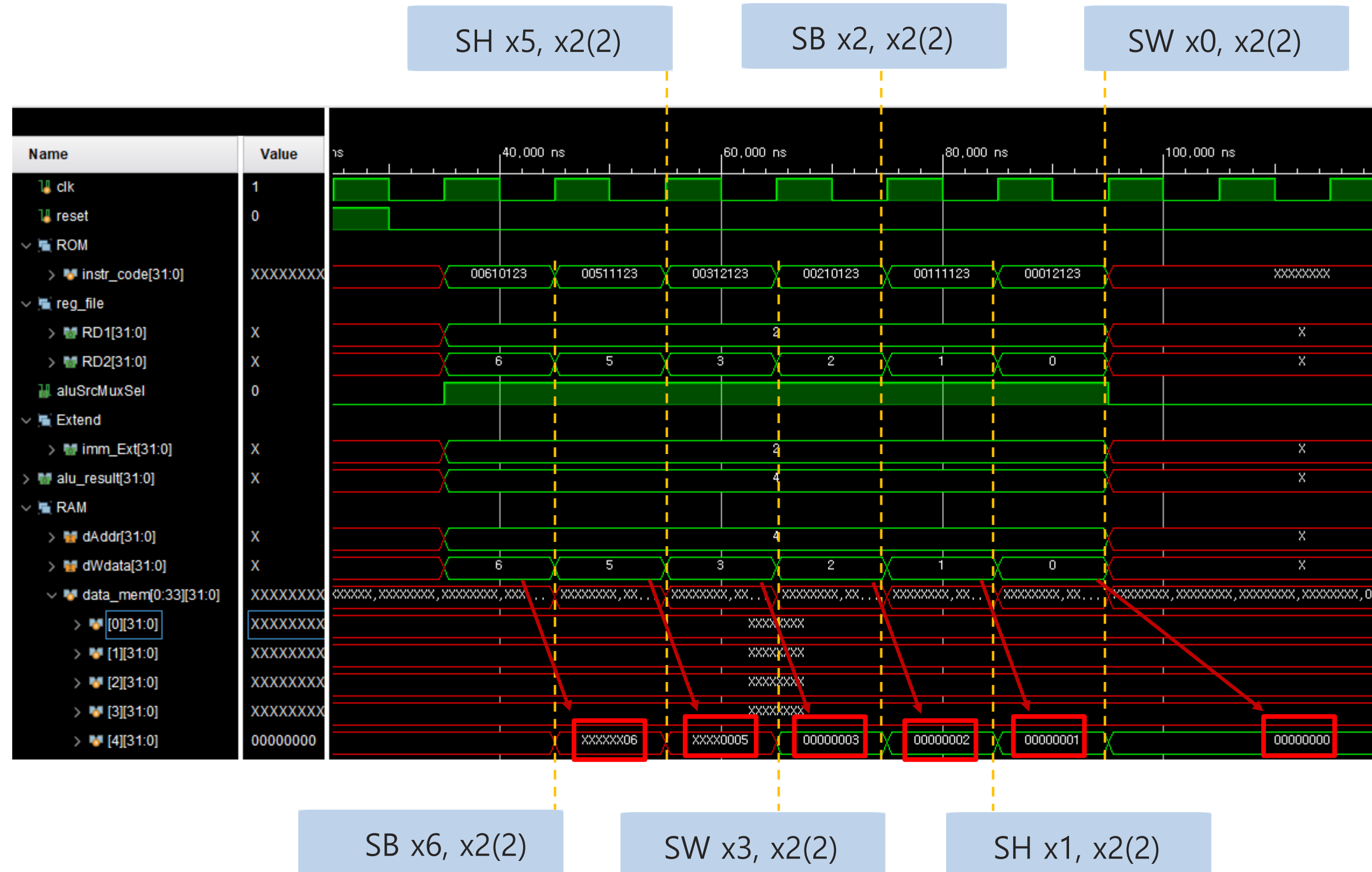
S-TYPE

- 역할 : 레지스터 값을 메모리에 저장
- 흐름 : Instruction_Memory에서 명령어 받음
 - > imm 확장 후 ALU로 주소 계산
 - > 해당 주소의 메모리에 레지스터 값 저장



CHAPTER 03

S-TYPE



- 레지스터(rs2)의 데이터를 주소(rs1 + imm)가 가리키는 메모리(RAM)에 저장
- SB (Store Byte) : 1바이트
- SH (Store Halfword) : 2바이트
- SW (Store Word): 4바이트

SB	$M[rs1+imm][0:7] = rs2[0:7]$
SH	$M[rs1+imm][0:15] = rs2[0:15]$
SW	$M[rs1+imm][0:31] = rs2[0:31]$

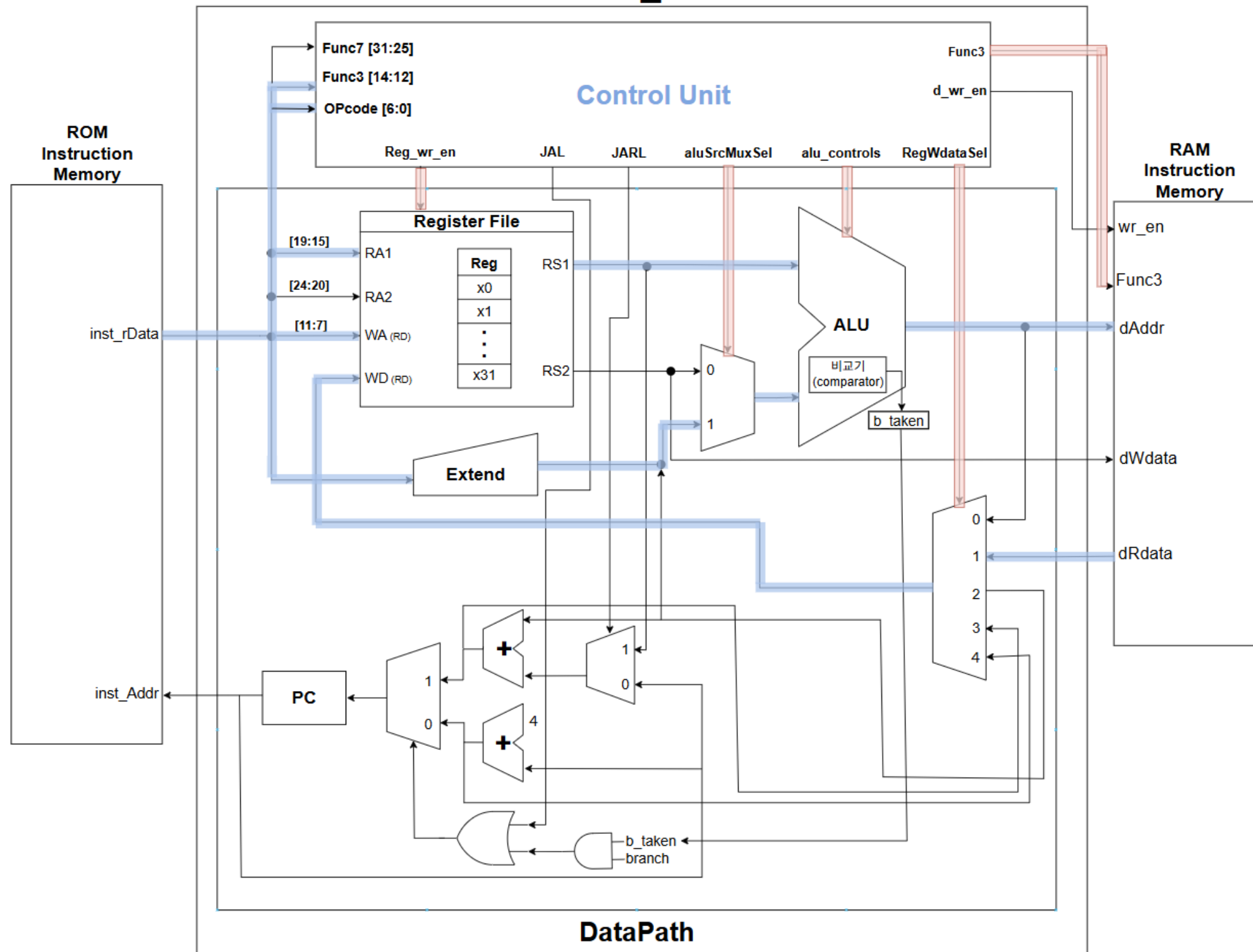


CHAPTER 02



I-TYPE (IL)

RV32I_Core

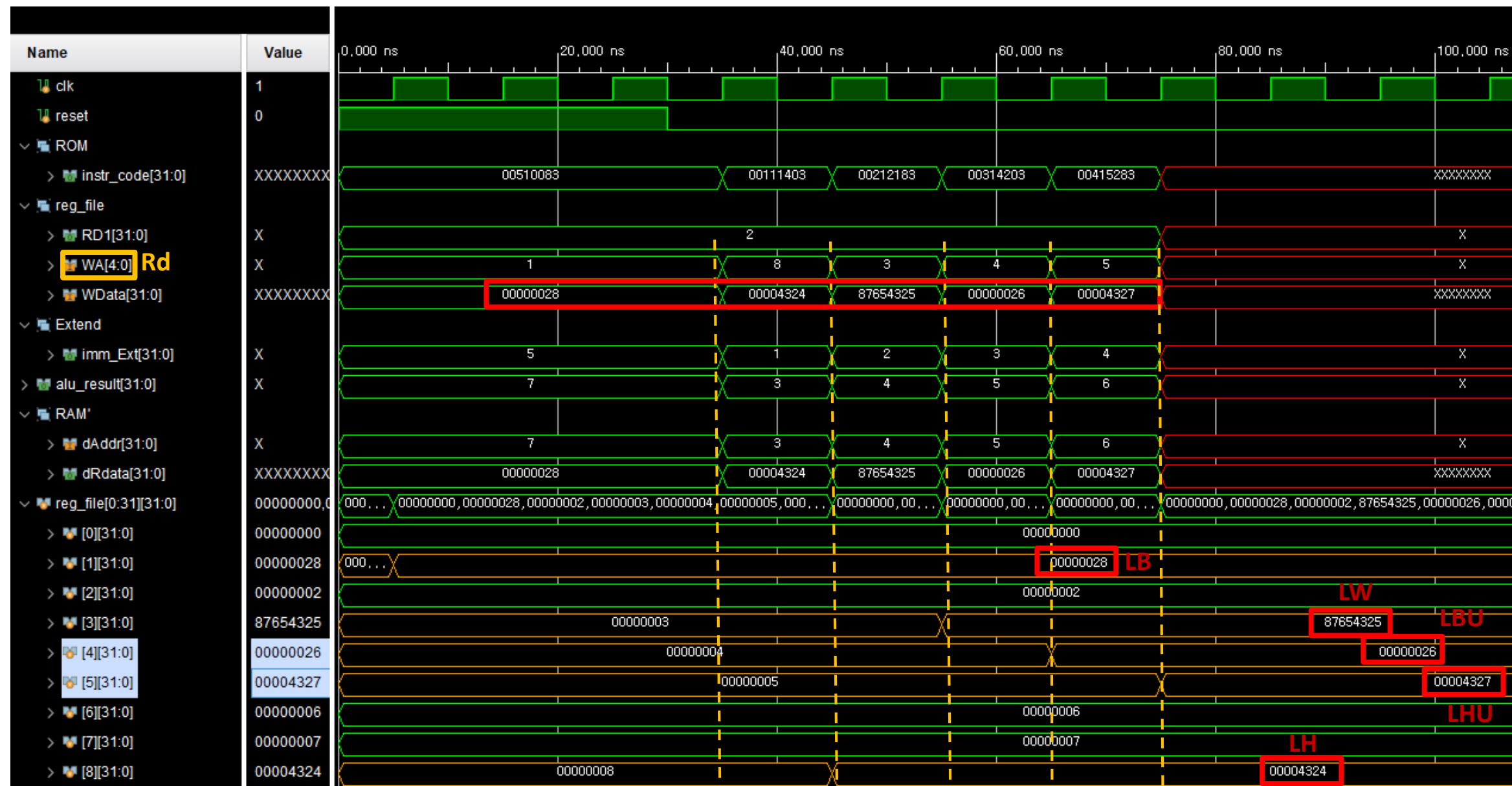


I-TYPE

- 역할 : 메모리에서 데이터를 읽어와 레지스터에 저장
- 흐름 : Instruction_Memory에서 명령어 받음
 - > imm 확장 후 ALU로 주소 계산
 - > 메모리에서 읽은 값이 레지스터에 저장

CHAPTER 03

I-TYPE (IL)



- 주소(rs1 + imm)가 가리키는 메모리에서 데이터를 읽어와 Rd 레지스터에 저장

	ROM (reg_file)	RAM	
7	7	8765_4328	7
6	6	8765_4327	6
5	5	8765_4326	5
4	4	8765_4325	4
3	3	8765_4324	3
2	2	8765_4323	2
1	1	8765_4322	1
0	0	8765_4321	0

```
initial begin
    for (int i = 0; i < 16; i++) begin
        data_mem[i] = i + 32'h8765_4321;
    end
end
```

LB x1, x2(5)

LW x3, x2(2)

LHU x5, x2(4)

LH x8, x2(1)

LBU x4, x2(3)

LB	rd = M[rs1+imm][0:7]	
LH	rd = M[rs1+imm][0:15]	
LW	rd = M[rs1+imm][0:31]	
LBU	rd = M[rs1+imm][0:7]	zero-extends
LHU	rd = M[rs1+imm][0:15]	zero-extends

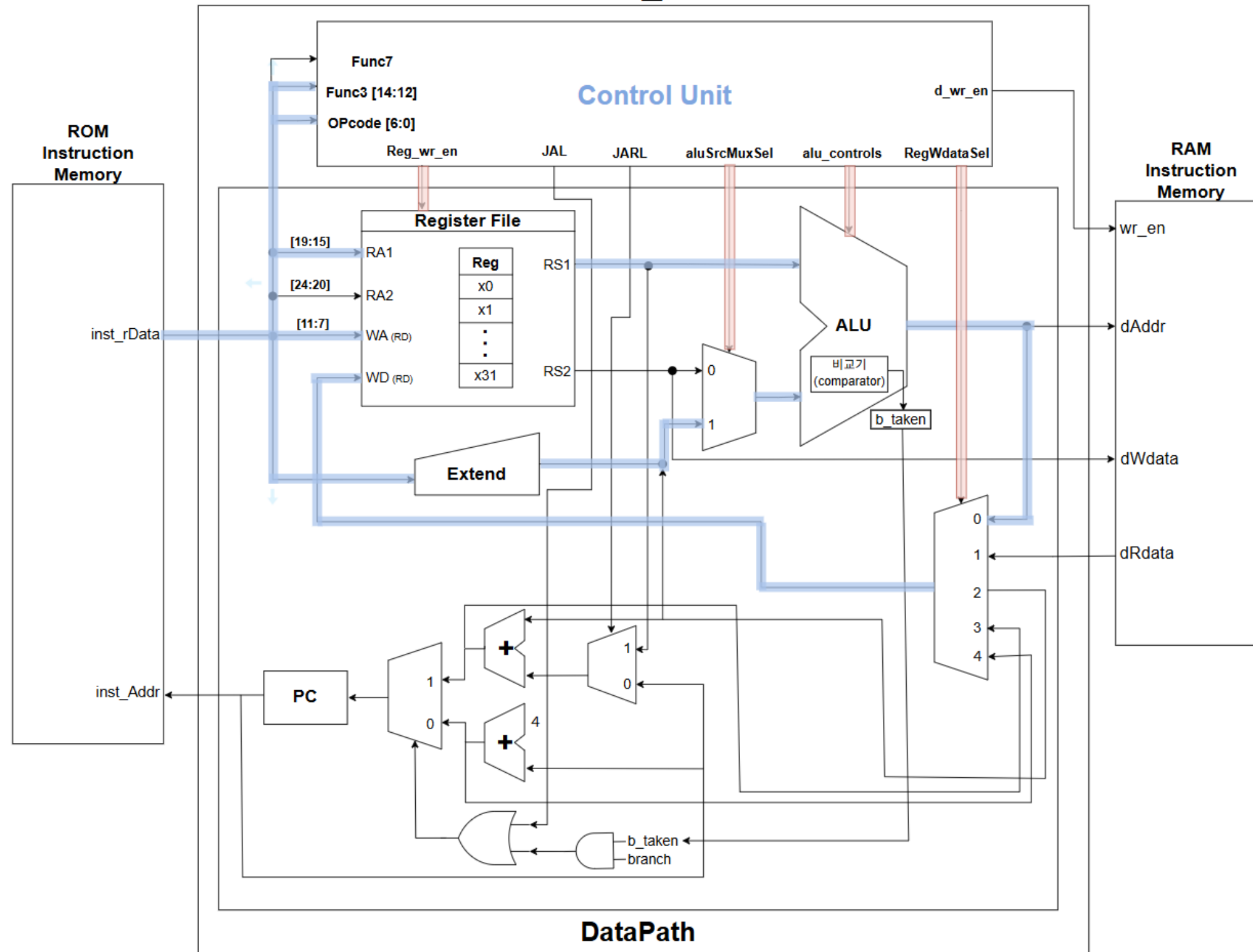


CHAPTER 02



I-TYPE

RV32I_Core



I-TYPE

- 역할 : 상수(Imm)와 레지스터 값을 이용해 연산
- 흐름 : Instruction_Memory에서 명령어 받음
 - > imm을 32bit로 확장해 ALU와 연산
 - > 결과가 레지스터에 저장



CHAPTER 03

I-TYPE

Addi x8, x2(12)

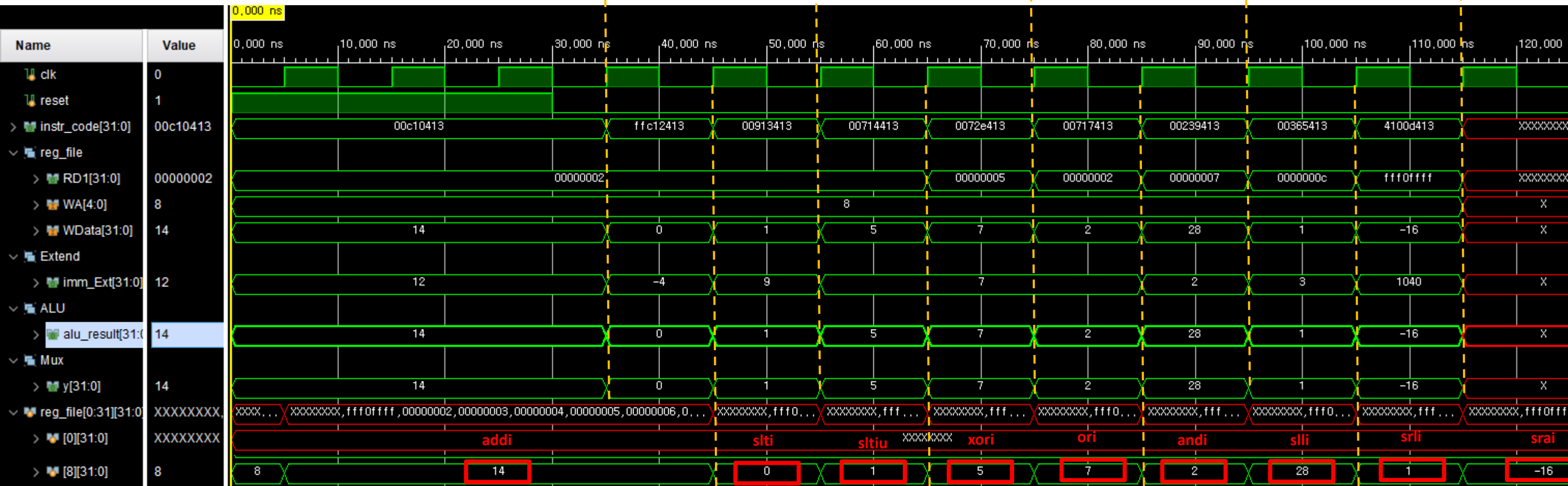
sltiu x8, x2(9)

ori x8, x5(7)

slli x8, x7(2)

srai x8, x1(16)

reg_file[1] = 32'b1111_1111_1111_0000_1111_1111_1111_1111;



- Rs1과 imm을 비교 / 연산한 결과를 rd에 저장

- Srai는 Msb extend이기 때문에
imm = 1040 =
010000010000 = 16 /
reg_file[1] = fff0_ffff를
16bit만큼 오른쪽으로
shift > rd에 -16 저장

slti x8, x2(-4)

xori x8, x2(7)

andi x8, x2(7)

srli x, 8x12(3)

ADDI	rd = rs1 + imm	
SLTI	rd = (rs1 < imm)?1:0	
SLTIU	rd = (rs1 < imm)?1:0	zero-extends
XORI	rd = rs1 ^ imm	
ORI	rd = rs1 imm	
ANDI	rd = rs1 & imm	
SLLI	rd = rs1 << imm[0:4]	
SRLI	rd = rs1 >> imm[0:4]	
SRAI	rd = rs1 >> imm[0:4]	msb-extends

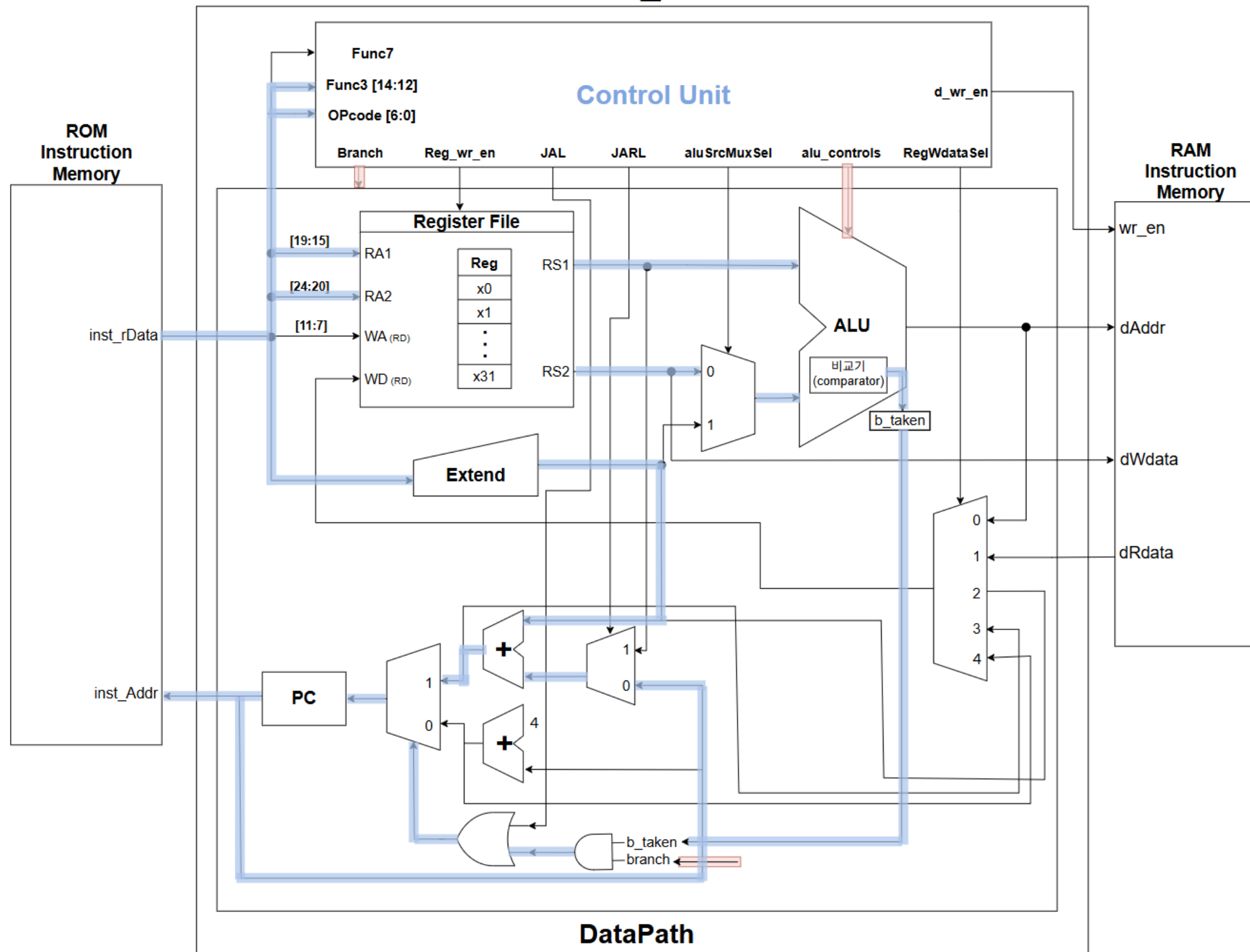


CHAPTER 02



B-TYPE

RV32I_Core



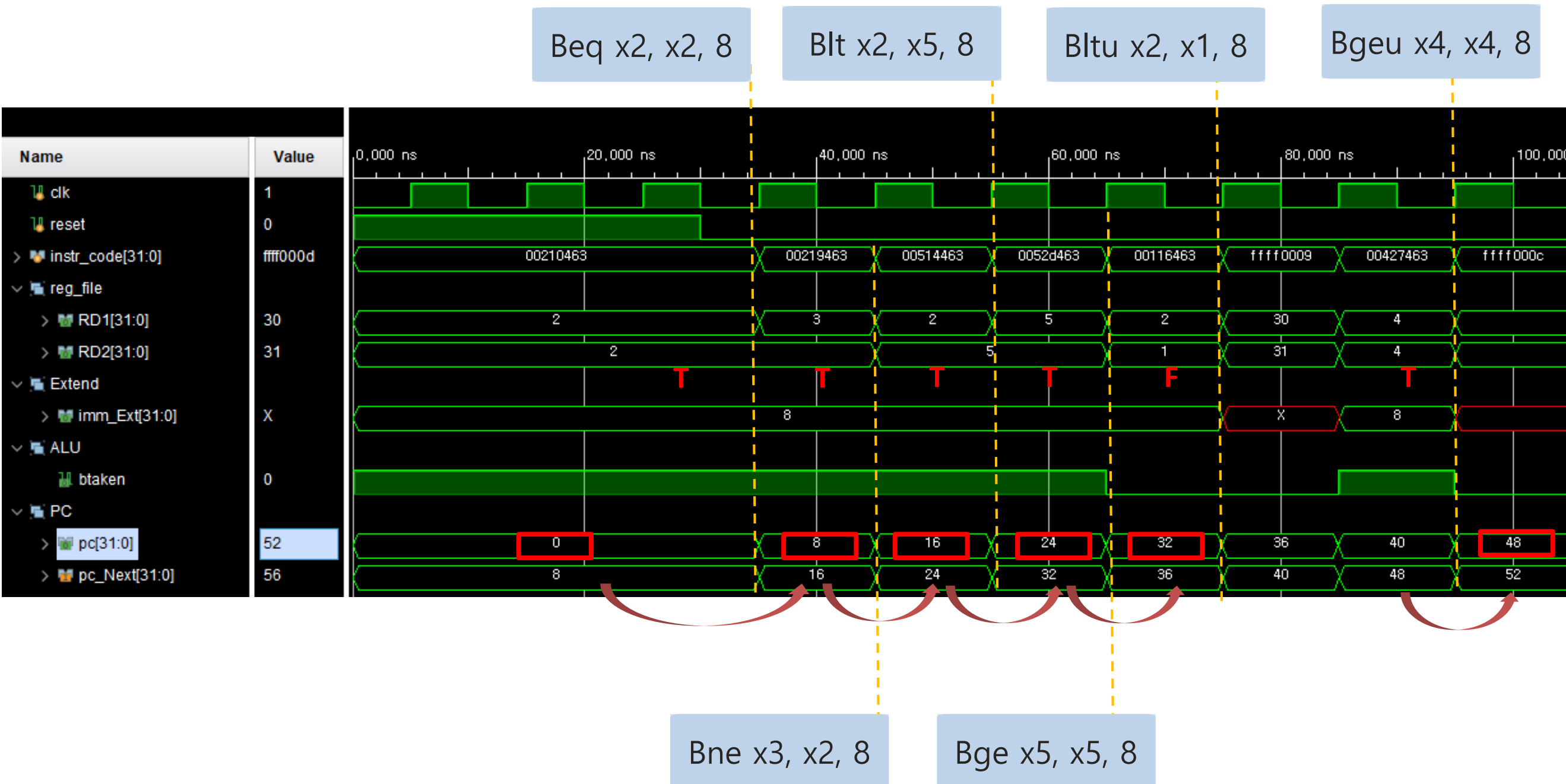
B-TYPE

- 역할 : 조건 분기를 수행
- 흐름 : Instruction_Memory에서 명령어 받음
 - > 두 레지스터 값 비교
 - > 조건이 참이면 PC가 imm만큼 이동



CHAPTER 03

B-TYPE



- 두 레지스터 (rs1, rs2)를 비교해 조건이 참이면 pc를 분기(pc+imm)

BEQ	if(rs1 == rs2) PC += imm	
BNE	if(rs1 != rs2) PC += imm	
BLT	if(rs1 < rs2) PC += imm	
BGE	if(rs1 >= rs2) PC += imm	
BLTU	if(rs1 < rs2) PC += imm	zero-extends
BGEU	if(rs1 >= rs2) PC += imm	zero-extends

- J 타입 & U 타입 사용이유 추가!!

- U 타입 : extend에서 주소를 32비트로 내보내기 위해 i타입의 addi와 함께 사용해줌
 - I타입에서는 addi를 통해 하위 12비트를 결정
 - 그리고 U 타입에서는 12비트 쉬프트 해줘서 상위비트 20비트의 주소를 결정해줌
 - 그렇게 해서 주소를 결정할 때 32비트를 전체 다 사용하기 위함임.
- J타입 : 함수를 호출하면 그 주소로 가는데 (jalr) 그 주소는 pC에 들어가고 그리고 레지스터에 다음 주소 저장
- Back할때 레지스터에 다음 주소 저장해둔 것 (jal)을 가져와서 그걸 pc로 주면 ? 다음 주소로 가게 됨

- C언어 > 어셈블리에서 어케 쓰는지?

- 리틀 엔디안인지 빅 엔디안인지도 ?

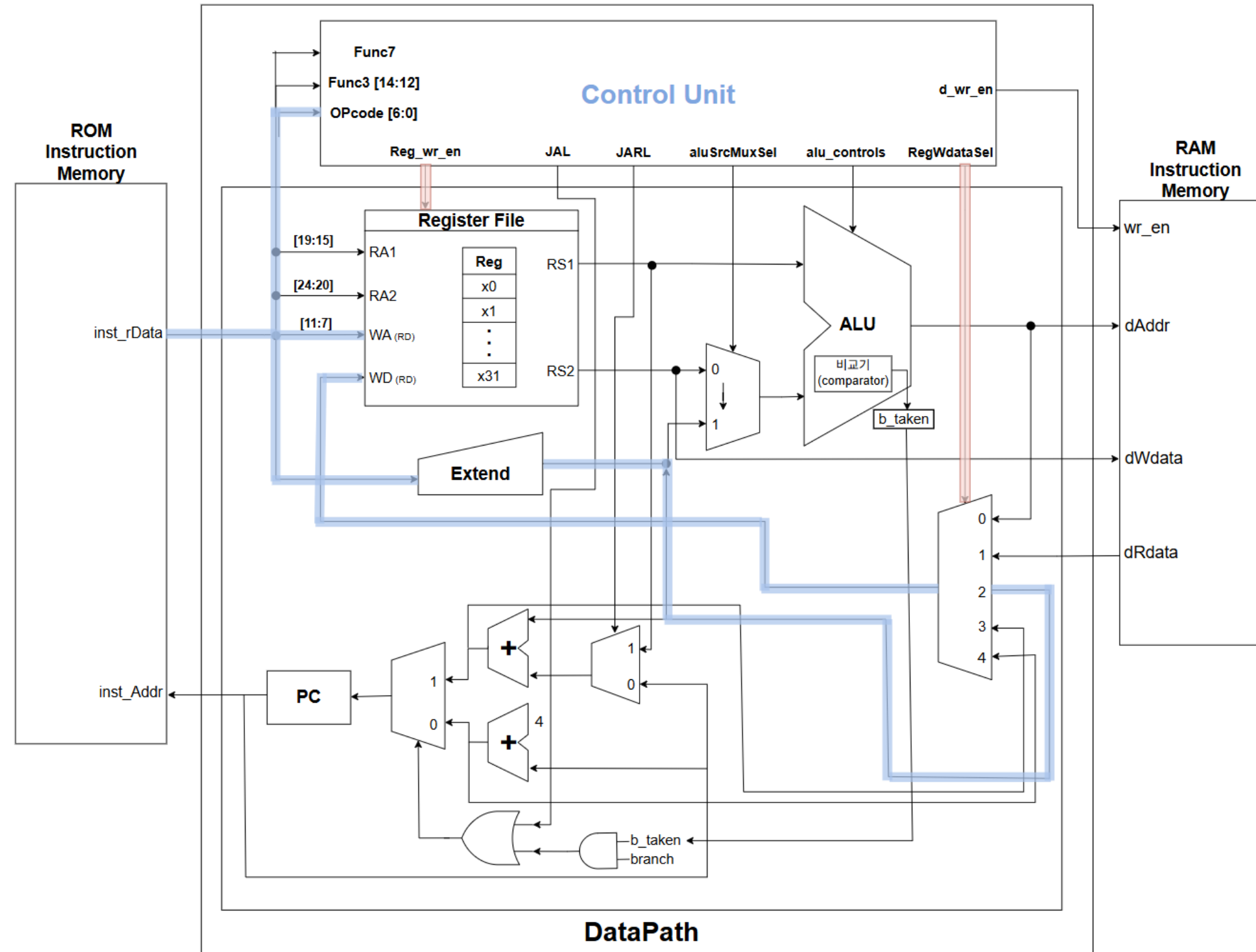


CHAPTER 02



U-TYPE (LUI)

RV32I_Core



U-TYPE

- 역할 : $\text{imm} \ll 12$ 를 레지스터에 직접 저장
- 흐름 : Instruction_Memory에서 명령어 받음
 - > 20bit 상수를 상위 비트로 확장
 - > 그대로 레지스터에 저장

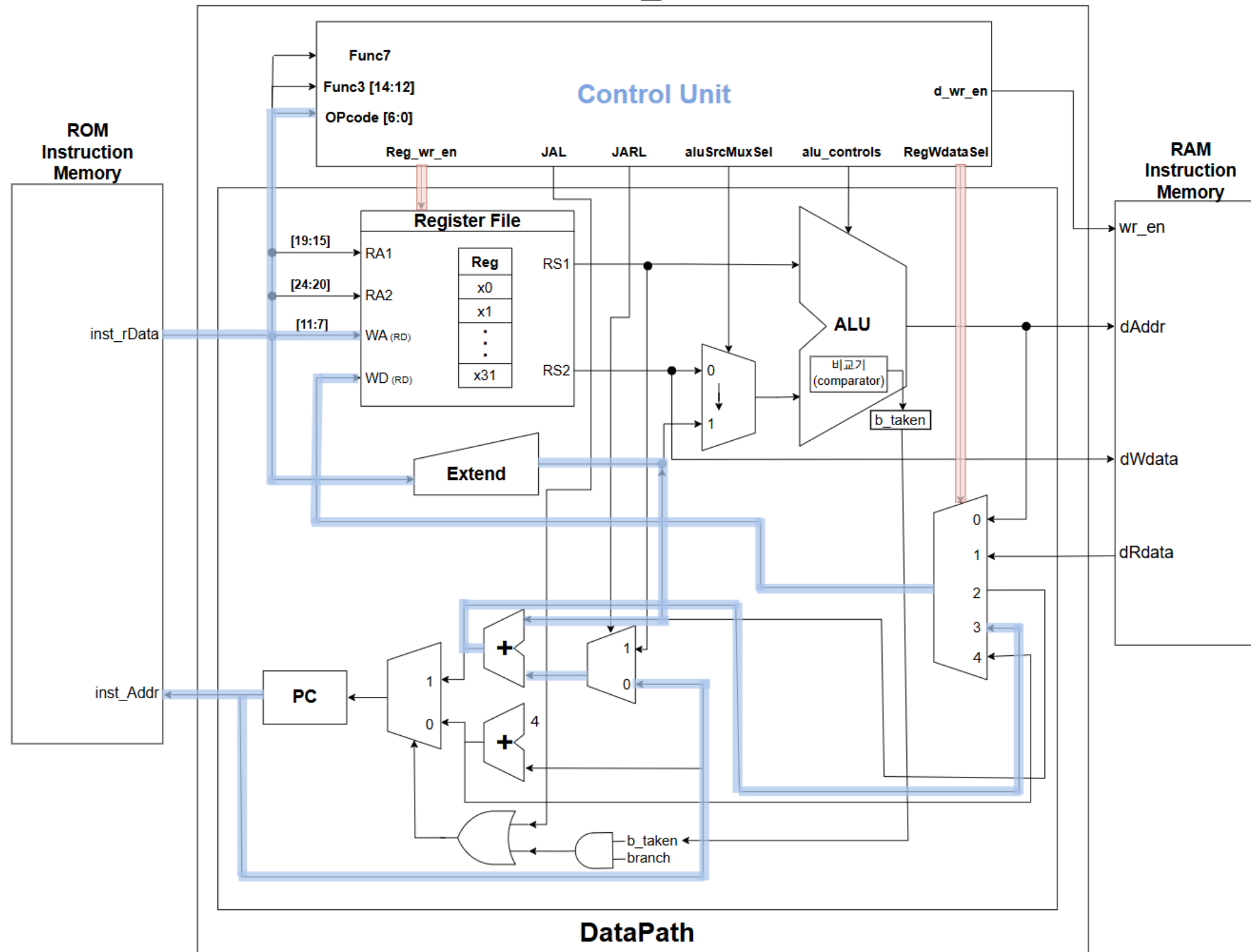


CHAPTER 02



U-TYPE (AUIPC)

RV32I_Core



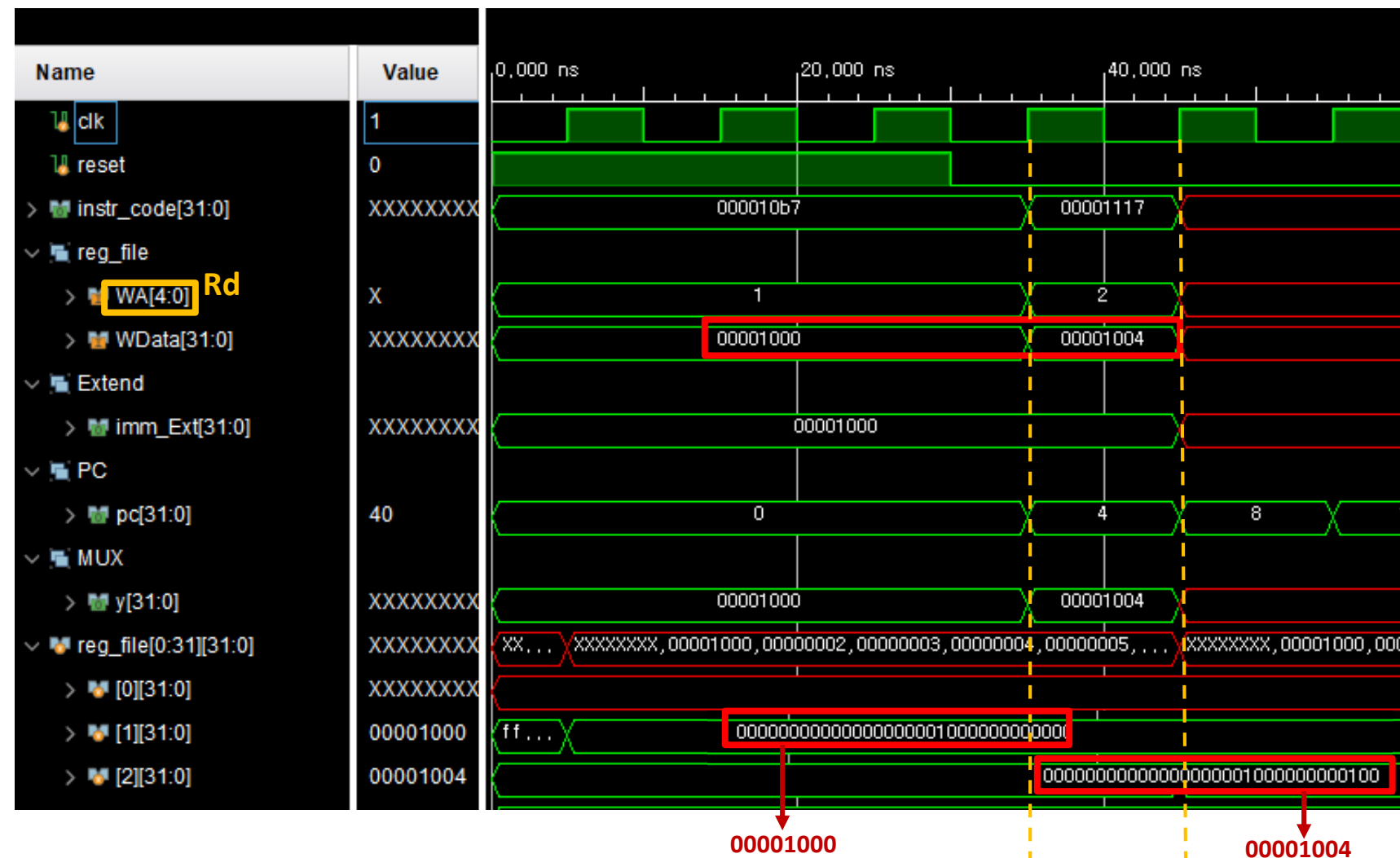
U-TYPE

- 역할 : 현재 PC와 $imm \ll 12$ 를 더해 주소를 계산
- 흐름 : Instruction_Memory에서 명령어 받음
 - > 20bit 상수를 32bit로 확장해 PC와 더함
 - > 결과를 레지스터에 저장



CHAPTER 03

U-TYPE



00001000

00001004

LUI
Imm = 1
Rd = x1

AUIPC
Imm = 1
Rd = x2

- LUI : $\text{imm} \ll 12$ 값을 rd에 저장
- AUIPC : $\text{imm} \ll 12$ 값에 pc를 더해서 rd에 저장
- LUI는 imm을 올려 0x1000을 저장하고, AUIPC는 현재 PC(=4)에 imm을 더해 0x1004를 저장

LUI	$\text{rd} = \text{imm}$
AUIPC	$\text{rd} = \text{PC} + \text{imm}$

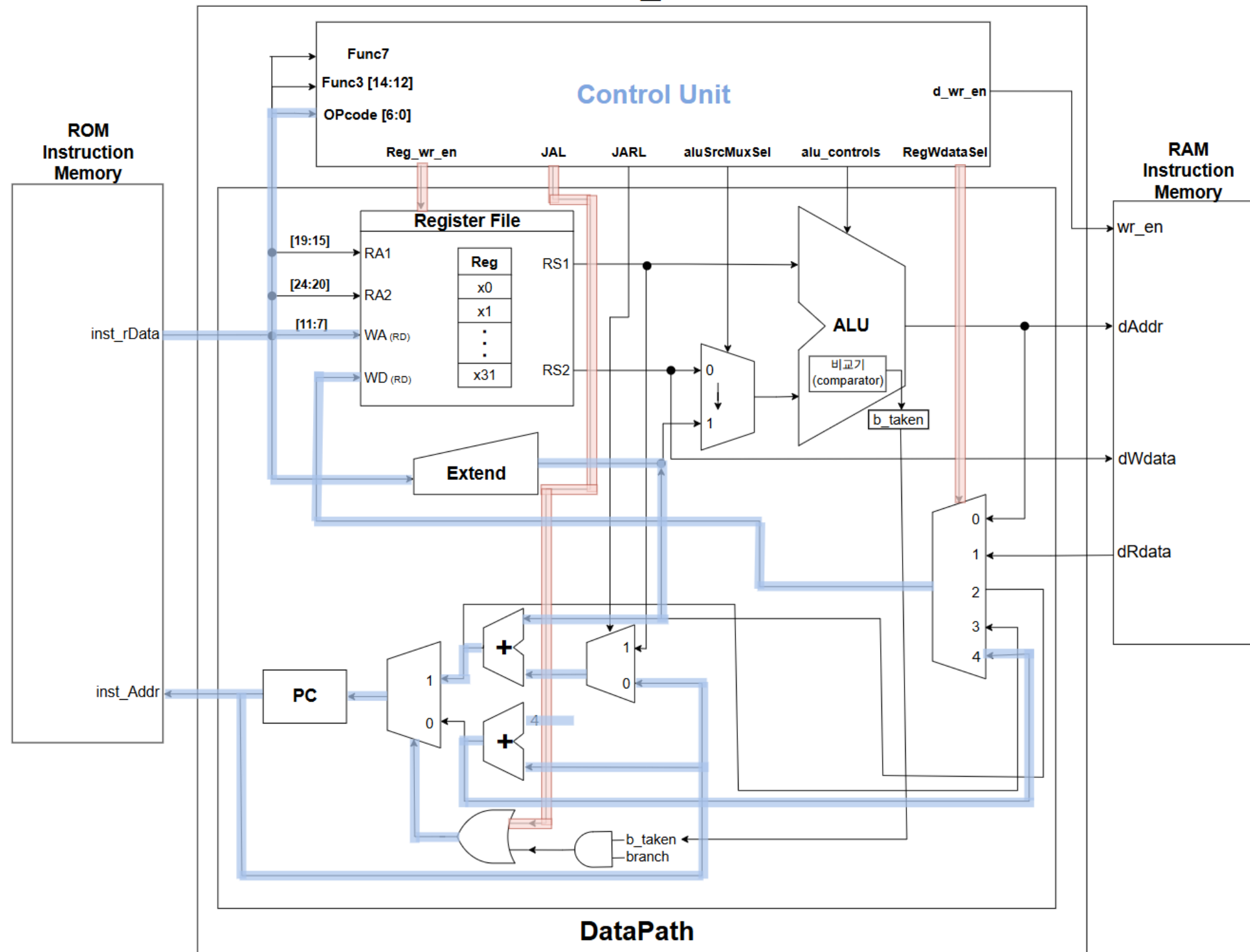


CHAPTER 02



J-TYPE (JAL)

RV32I_Core



J-TYPE

- 역할 : 무조건 점프하면서 복귀 주소를 함께 저장
- 흐름 : Instruction_Memory에서 명령어 받음
 - > imm 과 PC와 더한 주소로 점프
 - > 동시에 PC+4를 레지스터에 저장

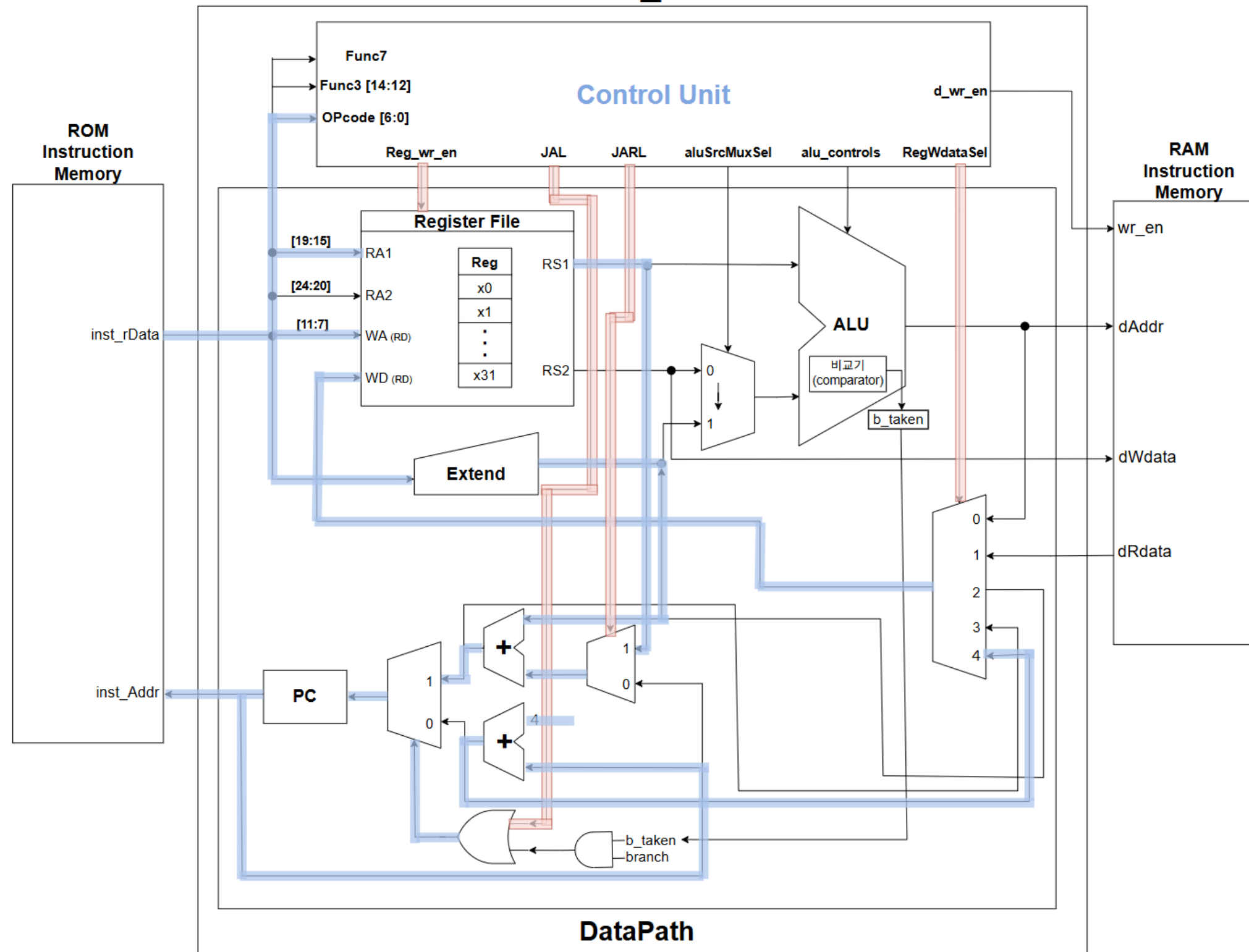


CHAPTER 02



J-TYPE (JALR)

RV32I_Core



J-TYPE

- 역할 : 레지스터 기반 무조건 점프와 복귀 주소 저장
- 흐름 : Instruction_Memory에서 명령어 받음
 - > (레지스터 값+imm)을 주소로 점프
 - > 점프하면서 PC+4를 레지스터에 저장

Discussion

RISC-V CPU 설계의 전반적인 과정을 경험

-> RISC-V ISA와 각 명령어의 동작 명확히 이해

시뮬레이션을 통하여 sign-extension이나 zero-extension과 같은
세부적인 동작까지 파형을 통해 직접 확인

-> RISC-V CPU Core 설계의 정확성을 높일 수 있음

Future Plans

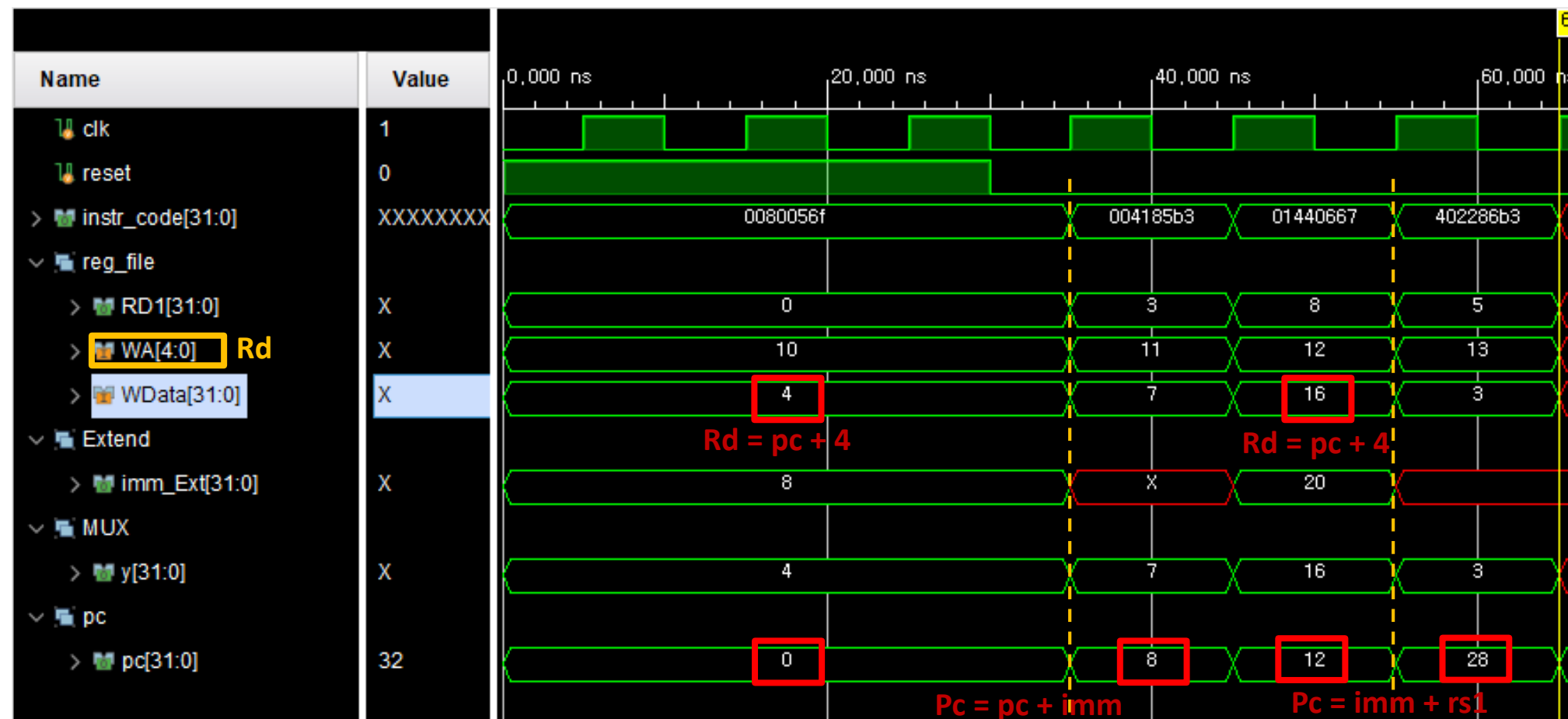
- Type 별 기능 검증
- 파이프라인 아키텍처 도입 -> CPU의 성능을 개선
(여러 명령어를 동시에 처리하여 전체 시스템의 명령어 처리량 높이기)

- 고찰 및 결론
- 실제 사용한 사례로 알게 된 것 혹은
- 어디를 틀렸는데 어떻게 고쳤는지 추가
- Extend를 연장어케하는지? 이게 헛갈렸음



CHAPTER 03

J-TYPE



JAL
Imm = 8
Rd = x10

JALR
Imm = 20
Rs1 = 8
Rd = x12

- JAL : PC+imm으로 점프하고, 복귀 주소(PC+4)를 rd에 저장
- JALR : rs1+imm 주소로 점프하고, 복귀 주소(PC+4)를 rd에 저장

JAL	$rd = PC+4; PC += imm$
JALR	$rd = PC+4; PC = rs1 + imm$



Reference

표1 : “VerilogHDL RISC-V 기본, RV32I R-Type, IL-Type,” *Minba’s blog*, 2024.06.18. → <https://minba-dev.tistory.com/entry/2-2>



Thank You